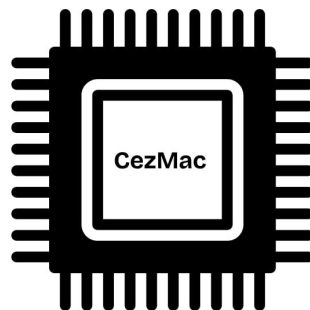
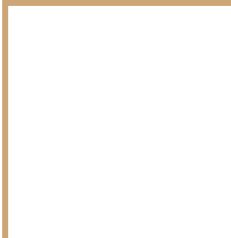


44h 5Ah 49h 45h 4Bh 55h 4Ah 45h



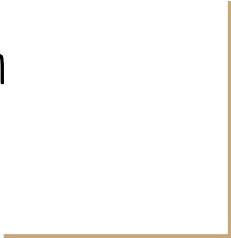
— 30.05.2026 —

📍 LUBLIN



# Warsztaty STM32

Zegarek binarny  
30.05.2026 Lublin



# Plan działania

1. Krótkie omówienie języka C oraz zestawu Nucleo G0B1RE
2. Podstawy transmisji szeregowej (UART)
3. Prosty projekt na rozgrzanie
4. Czym jest RTC?
5. Projekt zegarka binarnego
6. Synchronizacja zegarka z komputerem PC
7. Optymalizacja programu

# Projekt #0

Instalujemy lub uruchamiamy STM32CubeProgrammer

Podłączamy devboard do PC

Wrzucamy program "uartTest.hex"

Uruchamiamy Putty

Kopiujemy hasło do WiFi oraz link do repo z gita

Podziwiamy dzieło

# Materiały

**[https://github.com/CezMac/HandsOnSTM32\\_30052026](https://github.com/CezMac/HandsOnSTM32_30052026)**

-300 000

# Czy nauka programowania w dobie AI ma sens?

**Owszem ma!**

- Bielik AI – Polski model
- Mistral – Europejski model
- GitHub Copilot – dedykowany kodowaniu
- Codex – j.w. od OpenAI
- Claude Sonnet – Anthropic

# Czy matematyka jest potrzebna w programowaniu?

- Screen z gry Colobot - użycie funkcji trygonometrycznych (tangens) do obliczenia kątu strzału
- Różnego rodzaju filtry (A/B, Kalman), regulatory PID itp
- **Arytmetyka binarna (to dziś!)**
- **Logika**

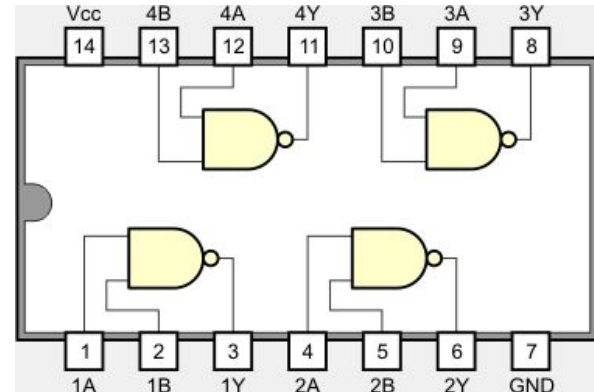


# Logika



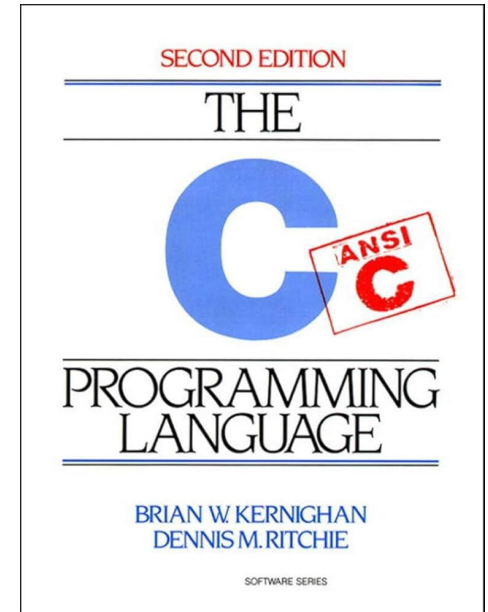
- Alternatywa  $P \vee Q$  (OR) ||
- Koniunkcja  $P \wedge Q$  (AND) &&
- Negacja  $\sim$  (NOT) !
- W STM32 wykorzystujemy logikę bramek logicznych
- Wykorzystując tylko zanegowane bramki AND (NAND) możemy stworzyć dowolną funkcję logiczną

| $p$ | $q$ | $p \wedge q$ | $p$ | $q$ | $p \vee q$ |
|-----|-----|--------------|-----|-----|------------|
| 1   | 1   | 1            | 1   | 1   | 1          |
| 1   | 0   | 0            | 1   | 0   | 1          |
| 0   | 1   | 0            | 0   | 1   | 1          |
| 0   | 0   | 0            | 0   | 0   | 0          |



# The C Programming Language

- Język C został stworzony w latach 70 przez Dennisa Ritchie i Kena Thompsona w Bell Labs, USA.
- Prosta i zwięzła składnia, niskopoziomowa integracja z sprzętem
- Język kompilowany
  - Instrukcje warunkowe (if-else if-else, switch, ==, !=, &&, ||)
  - Operacje bitowe (<<, >>, |, &, ^)
  - Operacje matematyczne (+ - / % \*)
  - Pętle (while, for, do-while)
  - Preprocesor (#define, #include, #ifdef)
- A dlaczego nie Python?



# C a C dla systemów wbudowanych

## **W zasadzie to brak różnicy!**

- W świecie systemów wbudowanych dąży się do unikania biblioteki standardowej
- Ograniczanie operacji zmiennoprzecinkowych (operacje stałoprzecinkowe)
- Ograniczanie dynamicznej alokacji pamięci (oprócz systemów czasu rzeczywistego lub tworzenie statycznych alokatorów pamięci)
- Ograniczone zasoby pamięci (RAM/FLASH)

# Programowanie PC a Mikrokontrolerów

- System operacyjny (tryb jądra/użytkownika)
  - Powłoka dla użytkownika (stdout/stdin)
  - Z reguły duża wydajność oraz uniwersalność
  - Obecnie wiele rdzeni
  - „Hello world!”
- Z reguły brak systemu (Bare-Metal, bezpośrednio dostęp do pinów MCU) lub RTOS, tryb Handler/Thread)
  - Brak domyślnej powłoki użytkownika (stdout/stdin)
  - Dedykowane konkretnemu przeznaczeniu
  - Głównie jeden rdzeń (ale to zależy!)
  - „Hello LED!”

# Typy zmiennych

PC (x86-64)

|                    |   |     |  |         |
|--------------------|---|-----|--|---------|
| short int          | : | 2 B |  | 16 bits |
| int                | : | 4 B |  | 32 bits |
| long int           | : | 8 B |  | 64 bits |
| long long int      | : | 8 B |  | 64 bits |
| short unsigned int | : | 2 B |  | 16 bits |
| unsigned int       | : | 4 B |  | 32 bits |
| long unsigned int  | : | 8 B |  | 64 bits |
| long long unsigned | : | 8 B |  | 64 bits |
| char               | : | 1 B |  | 8 bits  |
| float              | : | 4 B |  | 32 bits |
| double             | : | 8 B |  | 64 bits |
| bool               | : | 1 B |  | 8 bits  |

STM32 (ARM Cortex M0+)

|                    |   |     |  |         |    |
|--------------------|---|-----|--|---------|----|
| short int          | : | 2 B |  | 16 bits | LF |
| int                | : | 4 B |  | 32 bits | LF |
| long int           | : | 4 B |  | 32 bits | LF |
| long long int      | : | 8 B |  | 64 bits | LF |
| short unsigned int | : | 2 B |  | 16 bits | LF |
| unsigned int       | : | 4 B |  | 32 bits | LF |
| long unsigned int  | : | 4 B |  | 32 bits | LF |
| long long unsigned | : | 8 B |  | 64 bits | LF |
| char               | : | 1 B |  | 8 bits  | LF |
| float              | : | 4 B |  | 32 bits | LF |
| double             | : | 8 B |  | 64 bits | LF |
| bool               | : | 1 B |  | 8 bits  | LF |

| Typ kompilatora            | Typ jawny (zalecany)                        | Zakres wartości                    | Zastosowanie                                                                       |
|----------------------------|---------------------------------------------|------------------------------------|------------------------------------------------------------------------------------|
| short int                  | int16_t                                     | od -32768 do 32767                 | Obliczenia matematyczne                                                            |
| int                        | int32_t                                     | od -2,147,483,648 do 2,147,483,647 | Obliczenia matematyczne                                                            |
| long int                   | int32_t                                     | od -2,147,483,648 do 2,147,483,647 | Obliczenia matematyczne                                                            |
| long long int              | int64_t                                     | $-2^{63}$ do $2^{63} - 1$          | Obliczenia matematyczne                                                            |
| short unsigned int (Word*) | uint16_t                                    | od 0 do 65535                      | Liczniki, operacje bitowe                                                          |
| unsigned int (Word*)       | uint32_t                                    | od 0 do 4,294,967,295              | Liczniki, operacje bitowe                                                          |
| long unsigned int          | uint32_t                                    | od 0 do 4,294,967,295              | Liczniki, operacje bitowe                                                          |
| long long unsigned int     | uint64_t                                    | $2^{64} - 1$                       | Liczniki, operacje bitowe                                                          |
| Char (Byte)                | char, int8_t, uint8_t                       | 0 do 255 lub od -128 do 127        | char dla znaków ASCII,<br>int8_t obliczenia,<br>uint8_t liczniki i operacje bitowe |
| float                      | wymuszenie float<br><b>float var = 1.2f</b> | $\pm 3 \times 10^{38}$             | Obliczenia matematyczne                                                            |
| double                     | bez zmian                                   | $\pm 1.8 \times 10^{308}$          | Obliczenia matematyczne                                                            |
| bool                       | bez zmian                                   | True    False                      | Porównania logiczne                                                                |

# Dlaczego STM32?

- Popularne MCU w świecie elektroniki profesjonalnej/przemysłowej
- Darmowe środowisko programistyczne (STM32Cube IDE, VSC)
- Wydajne rdzenie z dużą ilością peryferii
- Przystępne cenowo zestawy startowe (Nucleo)
- **Kompatybilność z shieldami Arduino**
- Aktywne community
- Ale mamy też GD32, CH32 i inne!

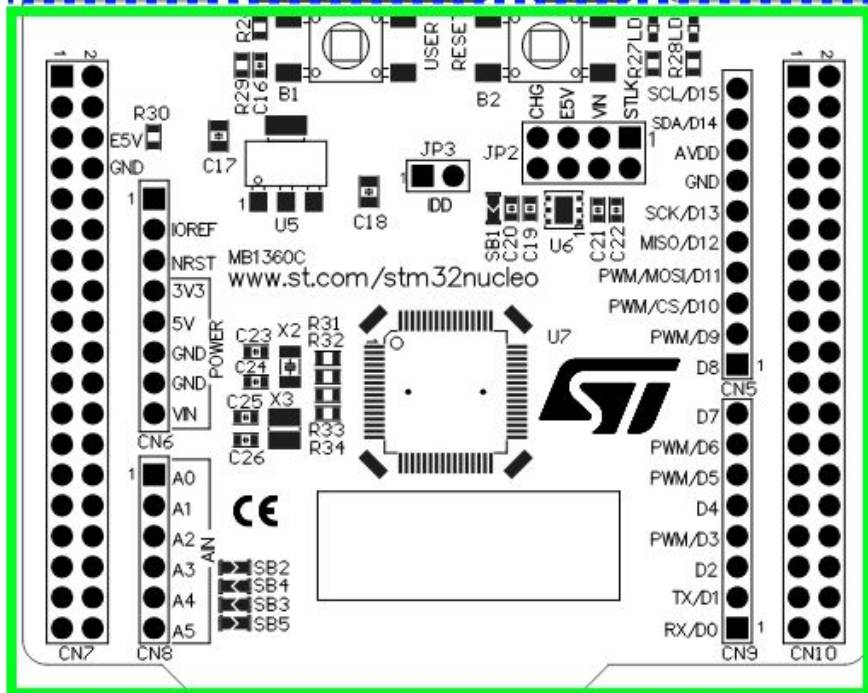
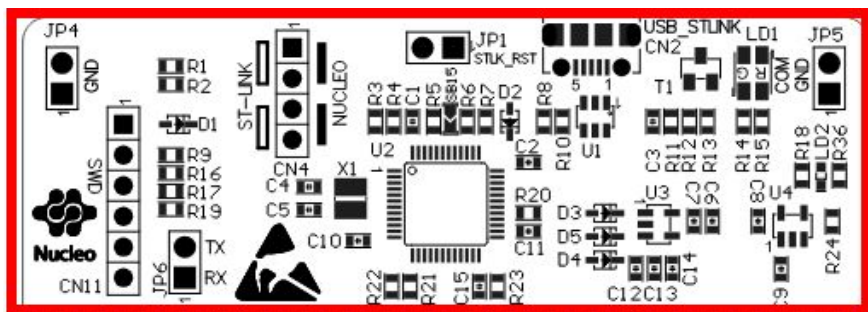
# Nucleo GOB1RE vs Arduino Uno R3

- Architektura ARM – 32bit (Cortex M0+)
- Zegar do 64 MHz
- Pamięć Flash 512 KB
- Pamięć RAM 144 KB
- Brak EEPROM (i tak i nie )
- 60 pinów I/O (3.3V)
- Wbudowany bootloader (UART/SPI/I2C/USB/CAN)
- NVIC/DMA
- Architektura AVR – 8 bit
- Zegar do 16 MHz
- Pamięć Flash 32 KB (4 KB dla bootloadera)
- RAM 2 KB
- EEPROM 1 KB
- 14 pinów I/O (5V - TTL)
- Ale jest świetne do uruchamiania nowych układów!

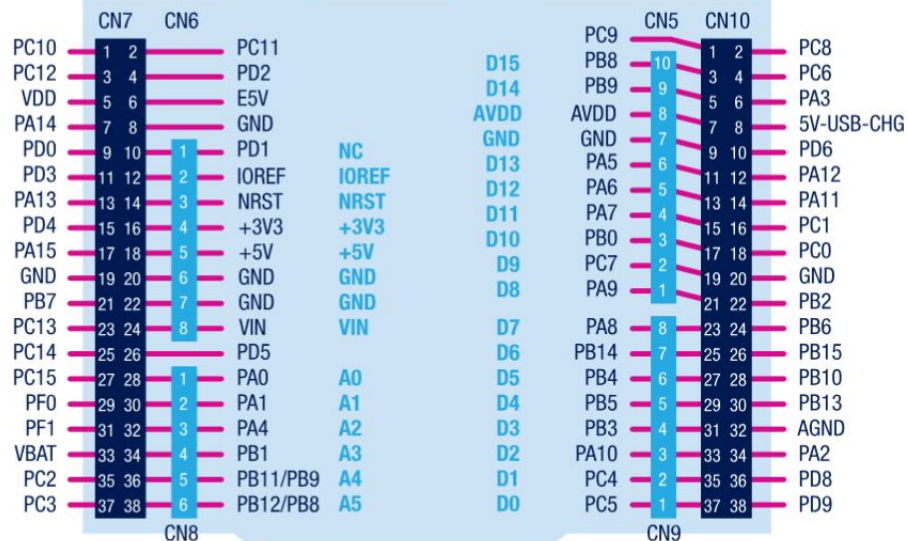
*Współczesne procesory są układami synchronicznymi ale są również **asynchroniczne**. Przykładem takiego procesora jest MERA400 produkowana w latach `70-`80 w zakładach MERA Warszawa*

# Nucleo STM32G0B1

| System                                                              | Arm® Cortex®-M0+ CPU<br>Up to 64 MHz         | Connectivity                                                        |
|---------------------------------------------------------------------|----------------------------------------------|---------------------------------------------------------------------|
| Power supply<br>POR/PDR/PVD/BOR                                     | Nested vector interrupt<br>Controller (NVIC) | 3x SPI (I²S)                                                        |
| Xtal oscillator<br>32 kHz + 4 to 48 MHz                             | SW debug                                     | 6x USART<br>(3x with LIN, smartcard,<br>IrDA, modem control)        |
| Internal RC oscillators<br>32 kHz (±5%)<br>+ 16 MHz (±1%)           | Memory Protection Unit                       | 2x LPUART                                                           |
| Internal RC oscillator<br>48 MHz (auto trimming on<br>ext. synchro) | AHB-Lite bus matrix                          | 3x I²C<br>Fast Mode Plus<br>(2x SMBus, PMBus)                       |
| PLL + Prescaler                                                     | APB bus                                      | 2x FDCAN                                                            |
| Clock control                                                       | Up to 512-Kbyte<br>Flash memory              | USB FS 2.0<br>Device (crystal less)<br>Host                         |
| RTC/AWU                                                             | Up to 144-Kbyte SRAM                         | USB Power Delivery<br>(incl. BMC + PHY)                             |
| Systick timer                                                       | 20-byte backup registers                     |                                                                     |
| 2x watchdogs<br>(independent and window)                            | Boot ROM                                     |                                                                     |
| 94 I/Os on 100 pins                                                 | 12-channel DMA                               |                                                                     |
| Cyclic redundancy check<br>(CRC)                                    |                                              |                                                                     |
|                                                                     | Analog                                       | Control                                                             |
|                                                                     | Temp. sensor                                 | 1x 32-bit timer                                                     |
|                                                                     | 1x 12-bit ADC SAR<br>16-channels / 2.5 MSPS  | 1x 16-bit Motor C. timer<br>$f_{MAX} = 128$ MHz<br>4 PWM + 3 compl. |
|                                                                     | 1x 12-bit DAC 2ch                            | 6x 16-bit timers<br>one with $f_{MAX} = 128$ MHz                    |
|                                                                     | 3x comparators                               | 2x Low-power timers                                                 |



## NUCLEO-G070RB



Arduino Uno

ST morpho

# Shield zegarka binarnego



Jest kilka błędów projektowych ;)

# Kod ASCII

System kodowania, który przypisuje wartości liczbowe od 0 do 127 do konkretnych znaków: liter, cyfr, znaków interpunkcyjnych oraz instrukcji sterujących (np. przejście do nowej linii).

**0–31(dec)** – (znaki niedrukowalne) np. znak 10 to *Line Feed* (nowa linia), a 13 to *Carriage Return*.

**32–64(dec)** – Spacja, cyfry (0–9) i popularne znaki specjalne.

**65–90(dec)** – Wielkie litery alfabetu łacińskiego (od A do Z).

**97–122(dec)** – Małe litery alfabetu łacińskiego (od a do z).

**128–255(dec)** – Rozszerzony zestaw ASCII, który zawiera znaki narodowe (np. polskie znaki diakrytyczne)

## Kod znaku c:

```
printf("%c 0x%X %d", 'c', 'c', 'c');
```

Wyjście: c 0x63 99

Tabela znaków:

**[https://www.promotic.eu/pl/pmdoc/Directio  
ns/TableASCII.htm](https://www.promotic.eu/pl/pmdoc/Directio<br/>ns/TableASCII.htm)**

# Projekt #1

- Stworzenie projektu
- Konfiguracja zegarów
- Konfiguracja peryferiów
- Konfiguracja USART2
- **Hello World!**
- Jak użyć printf?
- Import zewnętrznych bibliotek



# UART

- Uniwersalny **Asynchroniczny**  
Nadajnik Odbiornik
- Punkt-Punkt
- Full-duplex
- RS232 vs UART vs UART TTL
- RS485
- 8 N 1
- **Transparentna transmisja**

**UART**

(Universal Asynchronous Receiver Transmitter)

Low speed  
off-board  
full duplex  
**Usage :**  
Terminal  
Gps  
Modem ...

Peer-to-peer communication



[www.parlezhvoustech.com](http://www.parlezhvoustech.com)

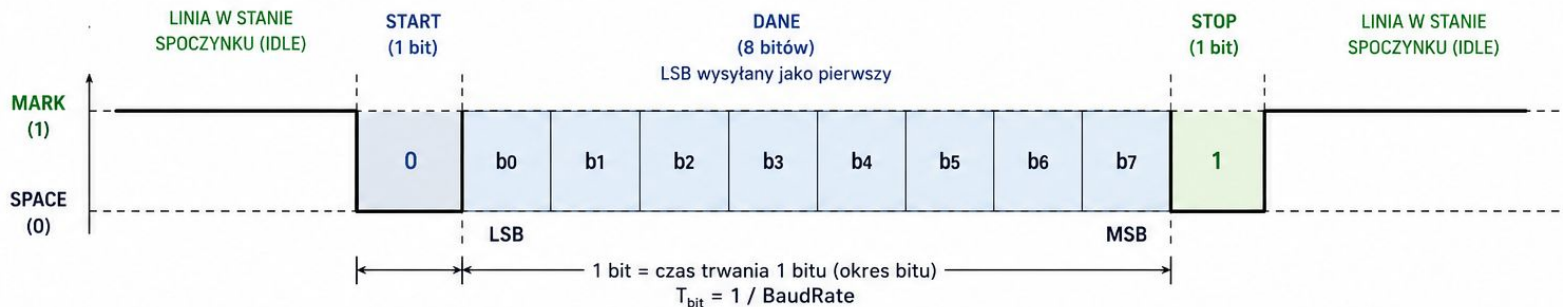
## UART 8N1 – BUDOWA RAMKI

UART (Universal Asynchronous Receiver-Transmitter) to popularny, asynchroniczny interfejs szeregowy.

8N1 oznacza: 8 bitów danych, N (None) – brak parzystości, 1 bit stopu

## 1. STRUKTURA RAMKI UART 8N1

Każda ramka składa się z 1 bitu startu, 8 bitów danych oraz 1 bitu stopu.

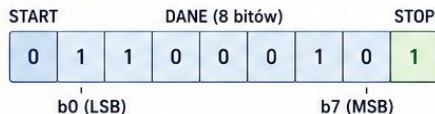


## 2. OPIS POLI RAMKI

|                       |                                                                                                              |
|-----------------------|--------------------------------------------------------------------------------------------------------------|
| <b>START (1 bit)</b>  | Zawsze 0. Informuje odbiornik, że zaczyna się nowa ramka. Przejście z 1 (idle) na 0 synchronizuje odbiornik. |
| <b>DANE (8 bitów)</b> | 8 bitów danych użytkownika (b0–b7). LSB (b0) wysyłany jest jako pierwszy.                                    |
| <b>STOP (1 bit)</b>   | Zawsze 1. Informuje o końcu ramki. Linia wraca do stanu spoczynkowego (idle = 1).                            |
| <b>PARZYSTOŚĆ</b>     | N – None. Brak bitu parzystości w trybie 8N1.                                                                |

### 3. PRZYKŁAD

Wyślijmy bajt: 0x53 (ASCII 'S')  
W zapisie binarnym: 0x53 = 0101 0011<sub>2</sub>  
LSB wysyłany pierwszy.



Ramka wystana (w kolejności czasowej):

0 | 1 1 0 0 1 0 1 0 | 1

#### 4. PARAMETRY TRANSMISJI



Asynchroniczna – brak wspólnego sygnału zegarowego.



Szybkość (BaudRate) – np. 9600, 115200.  
Określa liczbę bitów na sekundę.



**Format 8N1 – najczęściej używany:**  
8 bitów danych, brak parzystości, 1 bit stopu.



W stanie spoczynku linia = 1 (MARK).



**PAMIĘTAJ!**

W UART liczy się poprawne ustawienie tych samych parametrów (BaudRate, 8N1) po obu stronach – nadawcy i odbiorcy. Niezgodność parametrów powoduje błędne odbieranie danych.



UART

# UART

Obliczenie czasu transmisji ciągu znaków

$$t = N * 10 / \text{baud}$$

gdzie:

N - ilość znaków/bajtów, baud - prędkość np. 115200

“Ala ma kota” - 11 znaków

$$t = 11 * 10 / 115200$$

$$t = 9,548611111111111e-4 \sim 954 \text{ us}$$



# C STRING – CO TO JEST?

C string (łańcuch znaków) to sekwencja znaków typu char, zakończona specjalnym znakiem null `'\0'`.



## C STRING – CO TO JEST?

W języku C nie ma osobnego typu „string”.  
Łańcuch znaków to tablica znaków (`char[]`)  
zakończona znakiem null `'\0'`.  
Znak `'\0'` oznacza koniec łańcucha.

### Przykład

```
char tekst[] = "Hello";
```

↑  
kompilator dodaje `'\0'` na końcu



## JAK TO WYGLĄDA W PAMIĘCI?

Każdy znak zajmuje 1 bajt.

String musi kończyć się znakiem null (`'\0'`, czyli 0).

|       |               |     |     |     |     |                 |
|-------|---------------|-----|-----|-----|-----|-----------------|
|       | 0             | 1   | 2   | 3   | 4   | 5               |
| tekst | H             | e   | l   | l   | o   | <code>\0</code> |
|       | 72            | 101 | 108 | 108 | 111 | 0               |
|       | znaki stringa |     |     |     |     | znak null       |

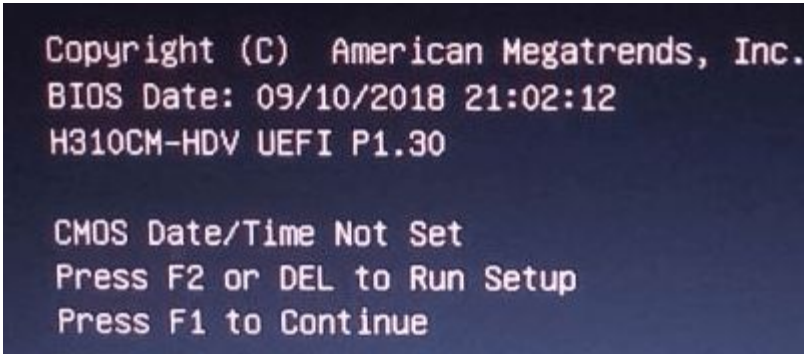


Funkcje z biblioteki standardowej wiedzą, że string się kończy, gdy napotkają `'\0'`.

# Projekt #2 - Zegarek binarny - RTC

**Zegar czasu rzeczywistego**, RTC (od ang. real-time clock) – element systemów komputerowych służący do odliczania czasu niezależnie od stanu maszyny (pracy, zablokowania, wyłączenia) *wg wikipedii*.

Z racji, że zegary RTC są niezależne od zasilania, to mają własną baterię podtrzymującą ich pracę.



```
Copyright (C) American Megatrends, Inc.  
BIOS Date: 09/10/2018 21:02:12  
H310CM-HDV UEFI P1.30  
  
CMOS Date/Time Not Set  
Press F2 or DEL to Run Setup  
Press F1 to Continue
```

# Projekt #2 - Zegarek binarny - RTC STM32

W STM32 RTC dzieli częstotliwość **LSE/LSI** 32 768 Hz przez  $2^{15}$  co daje nam 1s (MCU przesuwając liczbę 32768 o 15 -  $32768 \gg 15$ , co jest dla "dzieleniem" binarnym a tym samym wykonywane jest szybko).

RTC może precyzyjnie odmierzać:

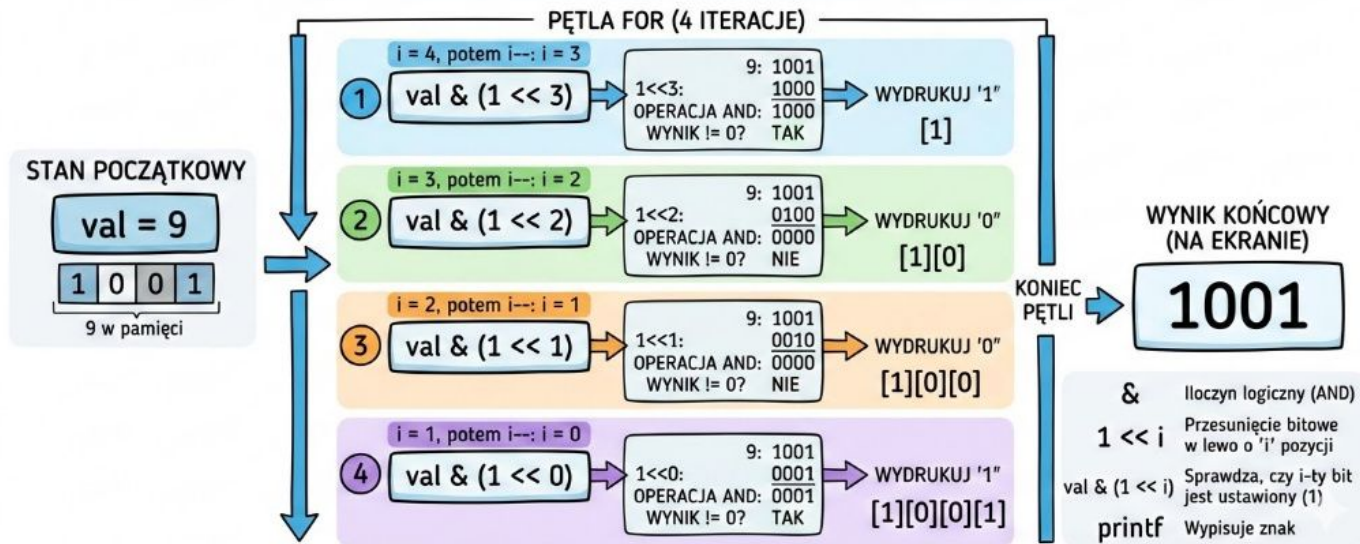
- sekundy,
- minuty,
- godziny,
- datę

RTC posiada rejestry "zapasowe" które potrafią przechowywać wartości zmiennych po resecie mikrokontrolera (ale tylko po resecie programowym! - chyba, że mamy podłączoną baterię)

# Projekt #2 - Zegarek binarny - Konwerter

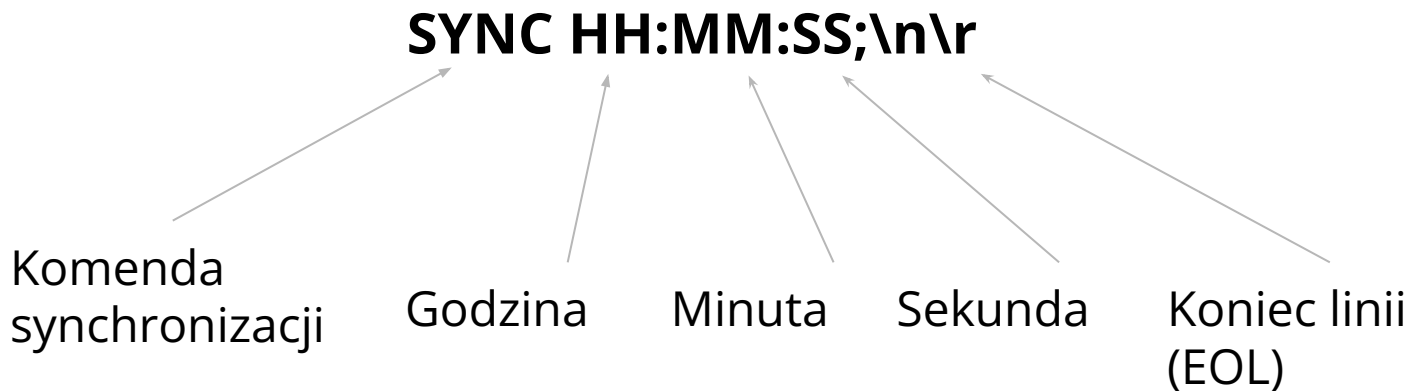
## JAK DZIAŁA TEN KOD C++? (KONWERSJA NA 4-BITOWY BINARNY)

```
int main()
{
    int val = 9;
    for(int i = 4; i > 0; i--)
    {
        if(val & (1 << i))
            printf("1");
        else
            printf("0");
    }
    return 0;
}
```



# Projekt #2 - Zegarek binarny - protokół

Protokół komunikacyjny pomiędzy STM32 a PC:



Komenda która ma dokładnie **14** znaków zostanie rozpoznana jako prawidłowa

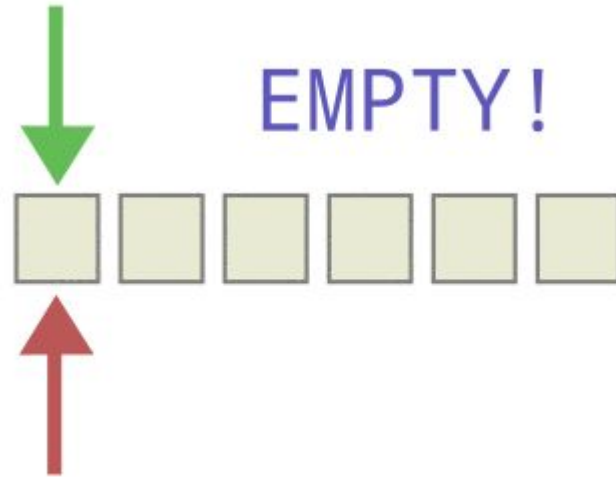
# Projekt #2 - Zegarek binarny - bufor kołowy

Bufor kołowy jest programową implementacją rozwiązania **Producent-Konsument**

Jedna strona jest **producentem** danych (zielona - head) które są wpisywane do bufora.

Druga strona jest **konsumentem** (czerwona - tail) czyli odbiera dane z bufora.

Odczyt i zapis jest realizowany ciągle. Dzięki temu po dojściu do końca bufora w czasie zapisu "przewijany" jest wskaźnik zapisu na początkową pozycję i nie musimy "relokować" lub czyścić tablicy podczas odczytu.



<https://hackaday.com/2015/10/29/embed-with-elliott-going-round-with-circular-buffers/>

# Projekt #2 - Zegarek binarny - Python

Program w Pythonie został przygotowany za pomocą **lokalnego** modelu AI

NVIDIA-Nemotron-3-Nano-4B-GGUF za pomocą programu LM Studio

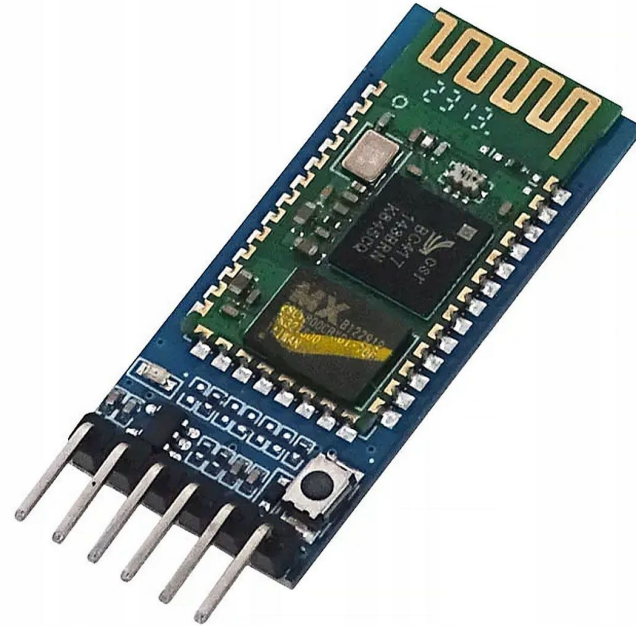
Zadaniem programu jest pobieranie czasu systemowego oraz wysłanie go przez port szeregowy do naszego zegarka

# Projekt # - Zegarek binarny - bezprzewodowy

Bluetooth (np. moduł HC05) - ale bluetooth ma ograniczony zasięg

LoraWAN - większy zasięg (np. DX2-LR02).  
Lora została zaprojektowana pod urządzenia IoT

Należy pamiętać o transparentnej transmisji danych czyli dane są wysyłane bez **szyfrowania!**



# Literatura

WYDANIE II

## JĘZYK

# C

ANSI  
C

## PROGRAMOWANIE

BRIAN W. KERNIGHAN  
DENNIS M. RITCHIE

NAJLEPSZA KSIĄŻKA  
O NAJPOPULARNIEJSZYM JĘZYKU!

Helion

## JĘZYK C W PROGRAMOWANIU URZĄDZEŃ

PRAKTYCZNA NAUKA TWORZENIA KODU  
DLA SYSTEMÓW WBUDOWANYCH

STEPHEN OUALLINE



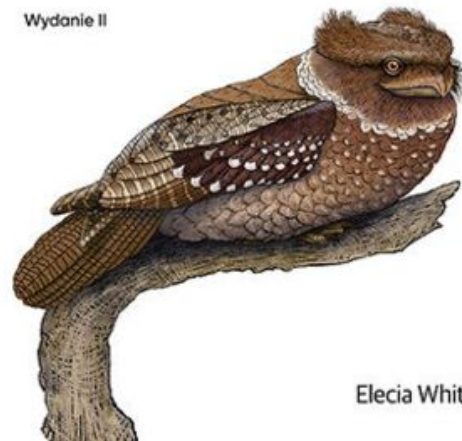
O'REILLY

Helion

## Systemy wbudowane

Wzorce projektowe  
dla twórców oprogramowania

Wydanie II



Elecia White

Polecane kursy



UCCG#SU



msalamon



MASTERS  
creative technology

# Polecane kursy

Najbliższe szkolenie:

<https://www.masters.com.pl/pl/workshops/schedule/schedule-preview/66/59>

Dziękuję za przybycie! :)